MLflow Experiment Tracking Report

Assignment 3 - CIFAR-10 Hyperparameter Tuning with MLflow

Ahmed | Datalentech | Spring 2025 MLOps

1. MLflow Runs Overview & Comparison View

The screenshot below shows all 5 experiment runs tracked in MLflow, displaying the Table View with runs sorted by best test accuracy, along with the Comparison View showing overlaid metric curves (test accuracy, test loss, train accuracy, train loss) across all runs.

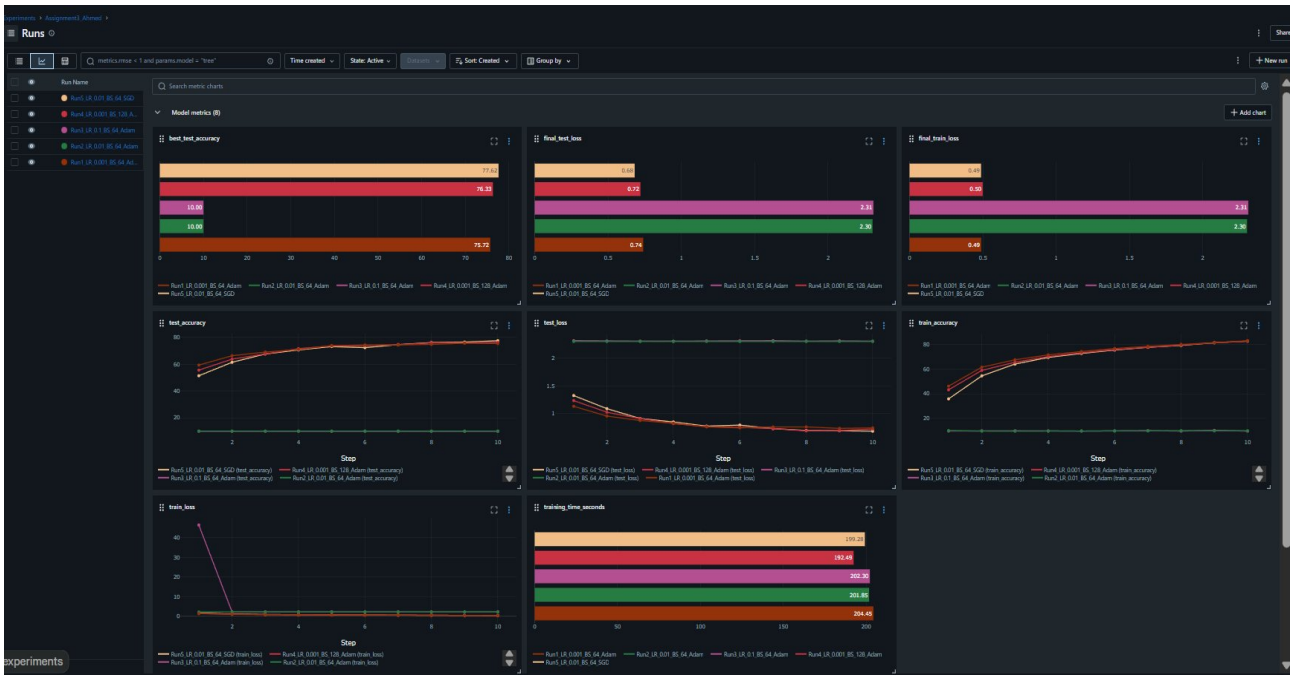

Figure 1: MLflow UI - Table View (sorted by best_test_accuracy) and Comparison View showing metric curves for all 5 runs, plus training time.

2. Artifact Gallery

The following screenshot shows the Artifacts tab for the best model, displaying the saved PyTorch model, the MLmodel metadata file, and auto-generated environment files (conda.yaml, python_env.yaml, requirements.txt).

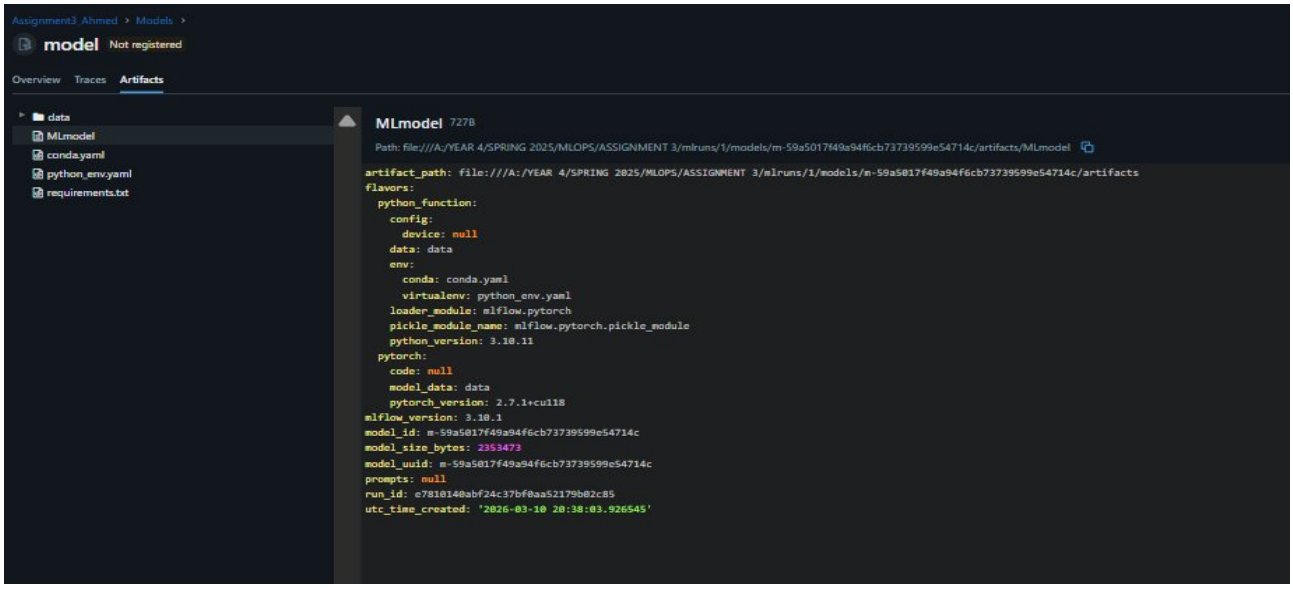

Figure 2: MLflow Artifacts tab showing saved model structure, MLmodel metadata (PyTorch 2.7.1+cu118, Python 3.10.11, MLflow 3.10.1), and environment files.

3. Experiment Runs Summary

Run	Learning Rate	Batch Size	Optimizer	Best Test Acc	Final Test Loss
Run 1	0.001	64	Adam	75.72%	0.740
Run 2	0.01	64	Adam	10.00%	2.304
Run 3	0.1	64	Adam	10.00%	2.310
Run 4	0.001	128	Adam	76.33%	0.721
Run 5	0.01	64	SGD	77.62%	0.682

Winner: Run 5 (LR=0.01, Batch Size=64, SGD) - highlighted in green.

4. Short Analysis

4.1 Why Run 5 Won

Run 5 achieved the highest test accuracy at **77.62%**, surpassing the runner-up (Run 4) by 1.29 percentage points. It also recorded the lowest test loss of **0.682**, indicating the best generalization to unseen data. Crucially, Run 5 had the smallest train-test gap of only **4.93%** (82.55% train vs 77.62% test), suggesting the least amount of overfitting among the successful runs.

4.2 Underfitting Evidence (Runs 2 & 3)

Both Run 2 (LR=0.01, Adam) and Run 3 (LR=0.1, Adam) exhibited severe underfitting, remaining stuck at approximately 10% accuracy throughout training. This is effectively random guessing for CIFAR-10's 10 classes. Their loss values plateaued at ~2.30, which equals ln(10) - the theoretical loss for uniform random predictions. Run 3 showed exploding gradients in its first epoch (train_loss = 46.5), confirming that a learning rate of 0.1 is far too aggressive for the Adam optimizer.

4.3 Mild Overfitting Evidence (Runs 1 & 4)

Runs 1 and 4 both used Adam with LR=0.001 and achieved reasonable performance (75.72% and 76.33% respectively). However, both exhibited classic signs of mild overfitting: training accuracy continued climbing while test accuracy plateaued or slightly declined in later epochs. Run 1's test accuracy dipped from 75.72% to 75.47% at the end, while both runs showed a widening gap between training and test metrics as training progressed.

4.4 Key Insights

Learning rate sensitivity: Adam failed catastrophically at LR >= 0.01, while SGD handled LR=0.01 effectively. This highlights the importance of matching the learning rate to the optimizer's characteristics.

Optimizer choice matters: SGD provided more stable and effective training than Adam at higher learning rates. The same hyperparameters (LR=0.01, BS=64) that completely failed with Adam produced the best overall result with SGD.

Batch size effect: With identical LR and optimizer settings, increasing batch size from 64 (Run 1) to 128 (Run 4) yielded a small improvement of 0.61% in test accuracy, suggesting that batch size has a moderate effect within this range.